

Herramientas computacionales para Economistas

Universidad Nacional de La Plata

Belén Michel Torino

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: concat y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: concat y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

En la clase pasada vimos:

- 1 Funciones
- 2 Manejo de errores
- 3 Módulos
- 4 Clases y objetos
- 5 Manejo básico de archivos

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: concat y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Objetivos de la clase

La clase de esta semana se centra en tres librerías centrales de Python:

- **NumPy** es una librería de cálculo numérico para Python, muchas de las principales librerías en análisis de datos están construidas sobre ella.
- **Pandas** es quizás la librería que más usen en sus primeros pasos en Python. Sirve para el trabajo con tablas de datos estructuradas, importar tablas de información, manipularlas y analizarlas.
- **Matplotlib**, la librería de base para graficar.

Hay mucho más para conocer sobre estas librerías de lo que se cubre en esta clase. El objetivo es que tengan el conocimiento suficiente para comenzar a usarlas y resolver las tareas más comunes.

Introducción a NumPy

NumPy es una librería de cálculo numérico para Python, lanzada inicialmente en 1995. Está escrita en lenguajes de bajo nivel como C y Fortran para permitir realizar operaciones matemáticas de manera eficiente.

Muchas librerías muy utilizadas en análisis de datos tienen a NumPy como librería base, y agregan funcionalidades adicionales por encima de ella. Algunos ejemplos son:

- Pandas: para manejo de datos tabulares,
- Matplotlib: para gráficos,
- SciPy: para cálculo científico,
- scikit-learn: para aprendizaje automático,
- TensorFlow y PyTorch: para aprendizaje profundo.

Las funcionalidades de esta librería que estudiaremos son:

- Arrays multidimensionales
- Operaciones numéricas básicas
- Operaciones de álgebra lineal
- Generación de números aleatorios

Como primer paso, importemos la librería (lo vimos en Clase 2):

```
1 >>> # primero importemos la libreria
2 >>> import numpy as np
3 >>> # definamos decimales en los resultados impresos
4 >>> np.set_printoptions(precision=3)
```

Arrays

Numpy está construido para trabajar de manera eficiente con vectores y matrices. Si bien un vector puede guardar información de manera similar que una lista, lo hace de manera más eficiente. Esta eficiencia en el almacenamiento y procesamiento de datos la hace más adecuada para procesamientos grandes.

```
1 >>> vector = np.array((1.,2,3,4))
2 >>> print(vector)
3 [ 1.  2.  3.  4.]
4 >>> vector.dtype
5 dtype('float64')
6 >>> print("vector ndim: ", vector.ndim)
7 vector ndim:  1
8 >>> print("vector shape:", vector.shape)
9 vector shape: (4,)
10 >>> print("vector size: ", vector.size)
11 vector size:  4
```

Arrays

Sigamos con otros ejemplos:

```
1 >>> matriz = np.array([[1,2,3], [4,5,6]])
2 >>> print(matriz)
3 [[1 2 3]
4  [4 5 6]]
5 >>> print("matriz ndim: ", matriz.ndim)
6 matriz ndim: 2
7 >>> print("matriz shape:", matriz.shape)
8 matriz shape: (2, 3)
9 >>> print("matriz size: ", matriz.size)
10 matriz size: 6
```

Arrays

Y para el último ejemplo, creamos un **Tensor**. Un tensor es una grilla de números, arrays, de n dimensiones. Un vector es un tensor de rango 1. Una matriz es un tensor de rango 2.

```
1 >>> # definimos un tensor de rango 3
2 >>> tensor3 = np.array([[[[-1,-2], [-3,-4]], [[0,0],
3 [[0,0]], [[2,1], [1,5]]]])
4 >>> print(tensor3)
5 [[[-1 -2]
6      [-3 -4]]
7      [[ 0  0]
8      [ 0  0]]
9      [[ 2  1]
10     [ 1  5]]]
11 >>> print("tensor3 ndim: ", tensor3.ndim)
12 tensor3 ndim:  3
13 >>> print("tensor3 shape:", tensor3.shape)
14 tensor3 shape: (3, 2, 2)
15 >>> print("tensor3 size: ", tensor3.size)
16 tensor3 size:  12
```

Operaciones on arrays

Similar a lo que vimos con operaciones entre variables, con Numpy es posible hacer operaciones entre arrays.

A continuación listamos algunos ejemplos:

```
1 >>> # creamos dos arrays
2 >>> a = np.array([[1, 2, 3, 4],[1, 2, 3, 4]])
3 >>> b = np.array([[4, 2, 2, 4],[4, 2, 2, 4]])
4 >>> # podemos realizar operaciones entre matrices
5 >>> a + b
6 >>> a - b
7 >>> a * b
8 >>> a / b
9 >>> a ** b
10 >>> a > 0.5
11 >>> 2 * a + 3
```

Números aleatorios

Por último, Numpy contiene un submódulo útil para crear números pseudo aleatorios.

```
1 >>> # importamos el submodulo
2 >>> from numpy import random
3 >>> # creamos un entero aleatorio entre 0 y 999
4 >>> x = random.randint(1000)
5 >>> # creamos un float aleatorio entre 0 y 1 (1 sin ser
incluido)
6 >>> y = random.rand()
7 >>> # inspeccionamos
8 >>> print(x, y)
9 >>> # creamos matrices aleatorias
10 >>> a = random.rand(2,3)
11 >>> print(a)
```

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: concat y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Objetivo: Trabajar con datos en formato estructurado. Refiere a tablas de datos como las que estamos acostumbrados a trabajar en Excel o Stata, información organizada en filas y columnas.

Pandas: es la librería más usada para este tipo de operaciones, tiene funciones y métodos que facilitan mucho el trabajo con datos tabulares.

Pandas define dos clases fundamentales:

- **Series:** similar a un `np.ndarray` unidimensional, pero los elementos tienen un índice asociado.
- **DataFrame:** la "tabla". Técnicamente, es una secuencia de **Series** donde el índice está compartido. En otras palabras, cada **Series** es una columna de la tabla, y el índice es un identificador para la fila.

En esta clase vamos a usar Pandas para:

- importar/exportar archivos de distintos formatos;
- explorar la tabla de datos;
- filtrar datos;
- agregar columnas / filas;
- unir bases (`merge`, `join`).

Abrir un dataframe I

Como primer paso, importemos la librería:

```
1 >>> import pandas as pd
```

Empecemos por abrir un archivo. Ya vimos una forma de hacerlo en la clase 2, ahora vamos a profundizar.

Por convención:

- Pandas se importa con el alias **pd**,
- la base importada se refiere como dataframe, y se abrevia **df**.

Pandas tiene funciones para abrir muchos tipos de archivos estructurados. Algunos ejemplos son:

- `pd.read_excel()`
- `pd.read_csv()`
- `pd.read_stata()`
- `pd.read_spss()`

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx")
```

Por defecto, Pandas abre la primera hoja del archivo (esté visible o escondida). Para indicar otra hoja:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Ahora, esa función abre el archivo pero no lo guarda en una variable. Para eso hay que asignarle un nombre de variable:

```
1 >>> import pandas as pd
2 >>> df = pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv("file_path.csv")
```

Al abrir un csv, read_csv asume que el separador es la coma. El separador es el símbolo que separa los valores: ",", ";", etc. Si el archivo usa otro, hay que especificarlo con sep:

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv("file_path.csv", sep=";")
```

Abrir un archivo de Stata es similar.

```
1 >>> import pandas as pd
2 >>> df = pd.read_stata("file_path.dta")
```

Nota: Pandas tiene documentación muy completa.

Crear un DataFrame

Veamos como crear un data frame vacío para luego agregarle filas.

```
1 >>> import pandas as pd
2 >>> df = pd.DataFrame(columns=["Nombre", "Apellido", "Curso",
    , "Profesor"])
```

Para agregarle contenido a mi tabla una opción sería agregarle una lista con los cuatro valores correspondientes a cada columna

```
1 >>> profesor1 = ["Belen", "Michel", "Python", True]
2 >>> profesor2 = ["Maria", "Sanchez", "Python", True]
3 >>> alumno = ["Sofia", "Tarufetti", "Python", False]
```

```
1 >>> df.loc[0] = profesor1
2 >>> df.loc[1] = profesor2
3 >>> df.loc[len(df)] = alumno
```

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Primero veamos las funciones de pandas que sirven para hacer una inspección general y una primera visualización de los datos importados:

- `df.head(N)`: nos muestra las primeras N líneas
- `df.tail(N)`: nos muestra las últimas N líneas
- `df.sample(N)`: muestra aleatoria de N líneas

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv("file_path.csv")
3 >>> df.head(5) # imprime nombres de las columnas y las 5
    primeras lineas
4 < OUTPUT SUPRIMIDO >
5 >>> df.tail(2) # igual que head pero empieza desde el fondo
6 < OUTPUT SUPRIMIDO >
7 >>> df.sample(10) #muestra aleatoria de n=10
8 < OUTPUT SUPRIMIDO >
```

Explorar los datos II

En los archivos de la Clase 3 (GitHub) hay un archivo `tabla_ejemplo.xlsx` que incluye registros inventados para grupos de estudiantes. Para cada grupo se cuenta con un identificador, la cantidad de integrantes y la edad promedio.

```
1 >>> import pandas as pd
2 >>> df = pd.read_excel("tabla_ejemplo.xlsx")
3 >>> df.head(2)
4      grupo  integrantes  edad_promedio
5 0        A           41       25.390012
6 1        B           55       39.969692
7 >>> # los 0 y 1 sobre el margen izquierdo son el numero de
      fila
8 >>> df.sample(3)
9      grupo  integrantes  edad_promedio
10 11      L           22       33.753149
11 8        I           52       25.543545
12 14      Z           50       36.578090
```

Explorar los datos III

Al importar un df, es una buena práctica imprimir algunas líneas para verificar que se haya cargado bien. También es útil imprimir el listado de nombres de las columnas y el tipo de dato:

- `df.columns` - listado de nombres de columnas,
- `df.dtypes` - tipo de dato por columna,
- `df.info()` - información más detallada sobre el DataFrame.

```
1 >>> #continuyendo con el df ya definido
2 >>> df.columns
3 Index(['grupo', 'integrantes', 'edad_promedio'], dtype='
      object')
4 >>> df.dtypes
5 grupo                str
6 integrantes          int64
7 edad_promedio        float64
8 dtype: object
```

Explorar los datos IV

Y para ver en más detalle:

```
1 >>> df.info(verbose=True) # verbose imprime resumen completo
2 <class 'pandas.DataFrame'>
3 RangeIndex: 15 entries, 0 to 14
4 Data columns (total 3 columns):
5 #      Column          Non-Null Count  Dtype
6 ---  -
7 0      grupo           15 non-null    str
8 1      integrantes      15 non-null    int64
9 2      edad_promedio    15 non-null    float64
10 dtypes: float64(1), int64(1), str(1)
11 memory usage: 492.0 bytes
```

Exportar I

Exportar también es sencillo con Pandas. Dos opciones comunes son:

- `df.to_excel()`
- `df.to_csv()`

Por default, el dataframe exportado se guardará en directorio en el que se está trabajando, con el nombre de archivo que especifiquemos.

```
1 >>> df.to_excel("exportar_ejemplo.xlsx", index=False)
2 >>> # listo!
```

Exportar II

Veamos si funcionó.

```
1 >>> df = pd.read_excel("exportar_ejemplo.xlsx")
2 >>> df.sample(1)
3      grupo  integrantes  edad_promedio
4 2      "C"           36         31.672932
5 >>> # Funciona! Ahora exportamos a csv
6 >>> # tambien podemos definir el separador
7 >>> df.to_csv("exportar_ejemplo.csv", index=False, sep="|")
8 >>> df = pd.read_csv("exportar_ejemplo.csv")
9 >>> df.columns
10 Index(['grupo|integrantes|edad_promedio'], dtype='str')
11 >>> # Una columna? Porque no especificamos el separador.
12 >>> df = pd.read_csv("exportar_ejemplo.csv" , sep="|")
13 >>> df.columns
14 Index(['grupo', 'integrantes', 'edad_promedio'], dtype='str',
      )
```

Trabajando con dataframes I

A continuación vamos a ver una serie de acciones que podemos realizar con dataframes.

- crear columnas
- cambiar tipos de datos
- aplicar funciones
- filtrar (slice)
- eliminar duplicados
- agregar (**append**)
- join/merge
- agrupar (**aggregate**)

Trabajando con dataframes II

Empecemos con cómo seleccionar columnas. Podemos hacerlo de dos maneras:

- `df["nombre_columna"]`: esto devuelve la columna como un objeto de tipo `pd.Series`. Se puede usar con nombres que incluyen espacios, sirve para crear columnas y operaciones con y entre columnas existentes. Es la que vamos a usar.
- `df.nombre_columna`: acceder como atributo de la dataframe. Una limitación es que no se puede usar con nombres que incluyen espacios, ni tampoco para crear columnas nuevas. Sí para operaciones con y entre columnas existentes.

Operaciones con columnas I

Un ejemplo:

```
1 >>> df = pd.read_excel("tabla_ejemplo_2.xlsx") #ejemplo2
2 >>> df.columns
3 Index(['grupo', 'inscriptos_ronda1', 'inscriptos_ronda2',
4        'edad_promedio', 'asignatura', 'area'], dtype='str')
5 >>> # df (nombre de la tabla) + enter, les imprime una
      muestra.
6 >>> # si df es grande, muestra las filas/columnas de los
      extremos, indicando con "..." las filas/columnas
      omitidas.
7 >>> # prueben poniendo df['asignatura'] y enter para
      visualizar
8 >>> # creamos una columna nueva con total de inscriptos
      llamada "inscriptos_total"
9 >>> df["inscriptos_total"] = df["inscriptos_ronda1"] + df["
      inscriptos_ronda2"]
10 >>> # impriman df o df["inscriptos_total"] para ver el
      resultado
```

Operaciones con columnas II

Continúa:

```
1 >>> # sumemos 2 inscriptos a cada grupo de la ronda 2 y
    volvamos a calcular el total
2 >>> df["inscriptos_ronda2"] = df["inscriptos_ronda2"] + 2
3 >>> df["inscriptos_total"] = df["inscriptos_ronda1"] + df["
    inscriptos_ronda2"]
4 >>> # generemos un id concatenando area y materia
5 >>> df["area_asignatura"] = df["area"] + df["asignatura"]
6 >>> # impriman el resultado
7 >>> # concatenado asi es confuso, incluyamos un separador
8 >>> df["area_asignatura"] = df["area"]+"_"+df["asignatura"]
9 >>> # impriman el resultado
10 >>> # generamos una columna que indique estado de
    inscripcion
11 >>> df["inscripcion_estado"] = "CERRADA"
12 >>> # impriman el resultado
```

Tipos de datos

Ya vimos cómo podemos usar Pandas para conocer el tipo de dato de cada columna, usando `df.info(verbose=True)`.

Para ver el tipo de dato de una columna podemos usar `df['columna'].dtype`:

```
1 >>> # inspeccionar el tipo de dato de una columna
2 >>> df["inscriptos_total"].dtype # singular, dtype != dtypes
3 dtype('int64')
4 >>> # con .astype() casteamos a otro formato
5 >>> df["inscriptos_total"] = df["inscriptos_total"].astype(
6     str)
7
8 >>> print(df["inscriptos_total"].dtype)
9 str
10 >>> # "str" de "string": ahora la columna es de texto
11 >>> # volvemos a formato int
12 >>> df["inscriptos_total"] = df["inscriptos_total"].astype(
13     int)
```

Aplicar funciones a columnas I

Las columnas son objetos iterables sobre los que podemos aplicar una función y afectará los registros en cada uno de los elementos del array.

Podemos aplicar funciones predefinidas como también funciones propias. Un ejemplo fue la función `df['columna'].astype()`. Otro ejemplo.

```
1 >>> # la funcion round() redondea al entero mas cercano
2 >>> df['edad_promedio_r'] = round(df['edad_promedio'])
```

También podemos usar `df['columna'].apply(funcion_x)`, donde `apply()` recorre la columna y ejecuta `funcion_x()` sobre cada elemento, uno por uno.

```
1 >>> # usando la misma funcion
2 >>> df["edad_promedio_r"] = df['edad_promedio'].apply(round)
```

Con `round`, las dos formas redondean igual: `round` acepta la Series completa. `.apply()` se vuelve necesario con funciones que esperan un único valor, como las que usan `if`.

Aplicar funciones a columnas II

Ahora un ejemplo con una función propia. Primero definimos la función:

```
1 >>> # function simple
2 >>> def clasificar_tamano(inscriptos):
3 ...     if inscriptos < 30:
4 ...         tamano = "chico"
5 ...     else:
6 ...         tamano = "grande"
7 ...     return tamano
```

Ahora la aplicamos:

```
1 >>> df['inscriptos_total'].apply(clasificar_tamano)
2 >>> df["tamano_clase"] = df['inscriptos_total'].apply(
    clasificar_tamano)
```

Seleccionar columnas

Para seleccionar un sub-conjunto de filas y/o columnas hay distintas alternativas. Para seleccionar columnas:

```
1 >>> # ya vimos como seleccionar una columna
2 >>> df["grupo"]
3 < OUTPUT SUPRIMIDO >
4 >>> # para seleccionar varias pasamos una lista
5 >>> df[["grupo","asignatura","inscriptos_total"]]
6 < OUTPUT SUPRIMIDO >
7 >>> # podemos guardar una copia como otro dataframe
8 >>> df_resumen = df[["grupo","inscriptos_total"]].copy()
```

Seleccionar filas I

Para seleccionar un sub-conjunto de filas podemos usar los índices:

```
1 >>> # para ver las filas 3 a 5
2 >>> df[3:6] # el limite superior no se incluye
3 >>> # para ver las primeras 3 filas
4 >>> df[:3]
5 < OUTPUT SUPRIMIDO >
6 >>> # para ver las ultimas 3 filas
7 >>> df[-3:]
8 < OUTPUT SUPRIMIDO >
```

También podemos usar condiciones:

```
1 >>> # slicing
2 >>> df[df["inscriptos_total"]<30]
3 < OUTPUT SUPRIMIDO >
4 >>> # y combinar condiciones
5 >>> df[(df["inscriptos_total"]<30)&(df["edad_promedio"]>40)]
6 < OUTPUT SUPRIMIDO >
```

Seleccionar filas II

Para filtrar usando condiciones:

- Si hay una condición: `df[condición]`
- Si hay más de una condición: `df[(condición1) & (condición2)]`
o `df[(condición1) | (condición2)]`

Dónde la condición es booleana (evalúa a True o False), de ser 2 o más, debe estar unida por un operador `&` o `|` (para AND o OR respectivamente). Si son más de dos condiciones presten atención a los paréntesis. Algo para tener en cuenta:

```
1 >>> # and!=& y or!=|, no son intercambiables:
2 >>> # esto:
3 >>> df[(condicion1)and(condicion2)] # incorrecto
4 >>> # NO es igual a esto:
5 >>> df[(condicion1)&(condicion2)] # correcto
6 >>> # y esto:
7 >>> df[(condicion1)or(condicion2)] # incorrecto
8 >>> # TAMPOCO es igual a esto:
9 >>> df[(condicion1)|(condicion2)] # correcto
```

Seleccionar filas y columnas

Para combinar filtros de filas y columnas:

- `df.loc[filas,columnas]`: usando el índice para identificar. Notar que a diferencia de lo que ocurre con slicing, al usar `loc` con enteros el límite superior SI se incluye.
- `df.iloc[filas,columnas]`: usando la posición (como entero) para identificar. En `.iloc` el límite superior NO se incluye.

Nota: muchas veces a etiqueta y la posición de una fila coinciden.

```
1 >>> # ejemplo .loc
2 >>> df.loc[3:4,["grupo","asignatura"]]
3     grupo asignatura
4 3      D    Historia
5 4      E     Fisica
6 >>> # ejemplo .iloc:
7 >>> df.iloc[3:5,[0,4]]
8     grupo asignatura
9 3      D    Historia
10 4      E     Fisica
```

Eliminar filas y columnas

Para eliminar filas y columnas podemos:

- 'pisar' el dataframe con la selección de las filas/columnas que SI queremos guardado una nueva copia sobre la anterior.
- usar la función `df.drop()`.

La función `.drop()` toma como argumento una lista de labels, y puede usarse para eliminar filas o columnas. Con el parametro 'axis' definimos si eliminar filas (`axis=0`, el default) o columnas (`axis=1`).

```
1 >>> # ejemplo eliminar filas
2 >>> df.drop([0,3,12]) # implicitamente llama axis=0
3 < OUTPUT SUPRIMIDO >
4 >>> # ejemplo eliminar columnas
5 >>> df.drop(["inscripcion_estado"],axis=1) # no guarda
6 < OUTPUT SUPRIMIDO >
7 >>> # df = df.drop(["inscripcion_estado"],axis=1)
8 >>> # o definir el parametro inplace=True
9 >>> df.drop(["inscripcion_estado"], axis=1,inplace=True)
```

Concat

Una ventaja de Python es la facilidad con la que se puede trabajar con multiples bases al mismo tiempo. Si tenemos un conjunto de bases que queremos agregar una debajo de la otra (EPH por ejemplo), podemos usar `pd.concat([df_a,df_b])`

```
1 >>> # cargemos una versión recortada de la base
2 >>> df_a = pd.read_excel("tabla_ejemplo_3a.xlsx") #exploren
      contenido
3 >>> len(df_a)
4 9
5 >>> df_b = pd.read_excel("tabla_ejemplo_3b.xlsx") #exploren
      contenido
6 >>> len(df_b)
7 9
8 >>> df = pd.concat([df\_a,df\_b]) # exploren para ver
      resultado
9 >>> len(df)
10 18
```

Merge I

Podemos usar las funciones `.join()` y `.merge()` para unir horizontalmente bases con un identificador común. Veamos el caso de `.merge()` (`join` es similar, lo pueden ver en la documentación de Pandas).

La función `merge` se aplica con la forma:

`df_a.merge(df_b,on=lista_columnas_id, how=tipo_join)`. Donde los tipos de `join` son:

- **left:** preserva todos los datos de la base izquierda y sólo los `matcheados` de la base derecha.
- **right:** preserva todos los datos de la base derecha y sólo los `matcheados` de la base izquierda.
- **outer:** preserva todos los datos (`matcheados` o no)
- **inner:** preserva los datos de filas con identificador común (`matcheados`).

Merge II

Retomando el ejemplo:

```
1 >>> # continuando con el ejercicio
2 >>> df.columns
3 Index(['grupo', 'inscriptos_ronda1', 'inscriptos_ronda2', '
    edad_promedio'], dtype='str')
4 >>> df_c = pd.read_excel("tabla_ejemplo_3c.xlsx") #exploren
    contenido
5 >>> df_c.columns
6 Index(['grupo', 'asignatura', 'area'], dtype='str')
7 >>> df = df.merge(df_c, on="grupo") # exploren para ver
    resultado
8 >>> df.columns
9 Index(['grupo', 'inscriptos_ronda1', 'inscriptos_ronda2', '
    edad_promedio', 'asignatura', 'area'], dtype='str')
```

Datos duplicados y reset index

Es una buena práctica inspeccionar los resultados después de hacer un concat o merge. Por ejemplo, luego del concat, la indexación de las filas quedó repetida ya que se preservaron los índices de las bases originales. Para arreglar esto:

```
1 >>> # luego del .concat()  
2 >>> df.reset_index(drop=True) # index original es descartado
```

Dependiendo de los datos que estemos usando, o si las bases no están bien curadas, realizar un merge puede generar duplicación de registros. En este caso lo simulamos. Para eliminar registros duplicados:

```
1 >>> df.tail(6)  
2 >>> df.drop_duplicates(inplace=True)  
3 >>> # como quedo el indice de las filas?  
4 >>> df = df.reset_index(drop=True)  
5 >>> df  
6 < OUTPUT SUPRIMIDO >
```

Aggregate I

Una operación frecuente con dataframes es agregar los datos, agrupando sobre un conjunto de variables y aplicando una función de agregación a otras. Con pandas esto se logra usando la función `.groupby()`, de la forma:

```
df.groupby(by=lista_columnas_agrupamiento).agg(dict_var_func)
```

Dónde `lista_columnas_agrupamiento` es una lista con los nombres de columnas sobre las que se agrupa, y `dict_var_func` es un diccionario de variable a agregar (llave) y la función con la que se agrega (valor).

Algunas de las funciones de agregación son:

- `sum`
- `mean`
- `count`
- `first` / `last`
- `min` / `max`

Aggregate II

```
1 >>> # seguimos con el mismo df ya definido
2 >>> df_agg = df.groupby(by=["area", "asignatura"]).agg({"
    inscriptos_ronda1": "sum", "inscriptos_ronda2": "sum", "
    edad_promedio": "mean"})
3 >>> # las variables de agrupamiento pasan a definir el
    indice
4 >>> df_agg.reset_index(inplace=True) # drop=False por
    default
5 >>> df_agg
6 < OUTPUT SUPRIMIDO >
```

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: concat y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Introducción a Matplotlib I

Matplotlib es la librería base de graficación, sobre la cual se montan otras librerías (como wrappers). Dentro de Matplotlib, usamos la dependencia "pyplot" que se instala con la librería. Por convención importamos así:

```
1 >>> # importar libreria grafica
2 >>> import matplotlib.pyplot as plt
3 >>> #"plt" siendo el nombre de convencion
```

Matplotlib genera los gráficos sobre dos objetos interrelacionados:

- **Figure:** la hoja en blanco, el recuadro que contiene el/los gráfico/s. En términos prácticos esto es ocurre detrás de escenas, pero es lo que permite dibujar el gráfico.
- **Axes:** el gráfico en si, los ejes y la información gráfica. La representación de la información sobre ejes.

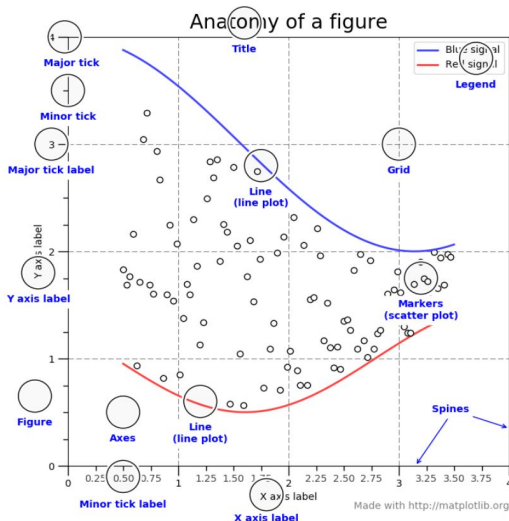
Introducción a Matplotlib II

¿Que podemos definir en un gráfico? Casi todo. Lo más que aprendamos lo más específicos que podemos ser (similar a Stata pero más flexible y con resultados superadores).

[Ver en la galería de ejemplos de Matplotlib](#)

Las partes de un gráfico:

Introducción a Matplotlib II



Fuente: Matplotlib User's Guide

Introducción a Matplotlib III

Hay esencialmente dos maneras de graficar con Matplotlib:

- **Estilo pyplot:** simple y rápida para figuras que no son muy avanzadas. Quizás más fácil para empezar.
- **Estilo orientado-objetos:** un poco más complejo pero necesario para figuras que requieren mucha customización.

En cuanto al resultado estético, con ambos se puede lograr la misma calidad. Para dar los primeros pasos es indistinto cual se use. Sin embargo, el estilo orientado a objetos es necesario para figuras más complejas dónde hay varios gráficos (subplots) y es necesario definir parámetros distintos para cada par de ejes (2D).

Preparar el ejemplo I

Poniendo en práctica lo que estuvimos aprendiendo, preparemos una nueva base:

```
1 >>> import pandas as pd
2 >>> # abrimos archivo de potencia energética instalada en el
    pais
3 >>> df = pd.read_excel("potencia_instalada.xlsx")
4 >>> # exploren la base. En la segunda hoja esta la fuente.
5 >>> # agregamos a nivel de tipo de fuente
6 >>> df_fuente = df.groupby(by=["periodo", "fuente_generacion"]
    ).agg({"potencia_instalada_mw": "sum"})
7 >>> df_fuente.reset_index(inplace=True)
8 >>> # exploren df_fuente
```

Preparar el ejemplo II

Vamos a graficar dos líneas, así que definimos vector X e Y para cada una. Vamos a graficar la potencia instalada de generación por fuente Renovable y fuente Térmica:

```
1 >>> # defino vectores de datos para serie 1
2 >>> y1 = df_fuente[df_fuente["fuente_generacion"]=="
    Renovable"]["potencia_instalada_mw"]
3 >>> x1 = df_fuente[df_fuente["fuente_generacion"]=="
    Renovable"]["periodo"]
4 >>> # defino vectores de datos para serie 2
5 >>> y2 = df_fuente[df_fuente["fuente_generacion"]=="Termica"
   ]["potencia_instalada_mw"]
6 >>> x2 = df_fuente[df_fuente["fuente_generacion"]=="Termica"
   ]["periodo"]
```

Nota: `df[condicion][columna]` selecciona la columna de la base que resulta de aplicar el filtro `df[condicion]`.

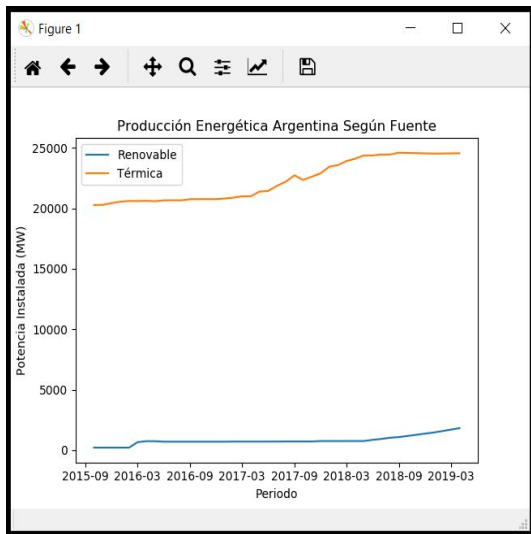
Graficar a la pyplot

Para graficar al estilo pyplot:

```
1 >>> import matplotlib.pyplot as plt
2 >>> # creo el grafico
3 >>> plt.plot(x1,y1, label='Renovable') # serie 1
4 >>> plt.plot(x2, y2, label='Termica') # serie 2
5 >>> #estas dos lineas estaran sobre el mismo grafico
6 >>> # modifico labels
7 >>> plt.xlabel("Periodo")
8 >>> plt.ylabel("Potencia Instalada (MW)")
9 >>> plt.title("Produccion Energetica Argentina Segun Fuente"
10 >>> # agrego leyenda
11 >>> plt.legend()
12 >>> plt.show() #esto es necesario para visualizar
```

Nota: luego de correr `plt.show()` se abre una ventana con el gráfico. Es necesario cerrarla para seguir trabajando sobre la consola.

Graficar a la pyplot: resultado



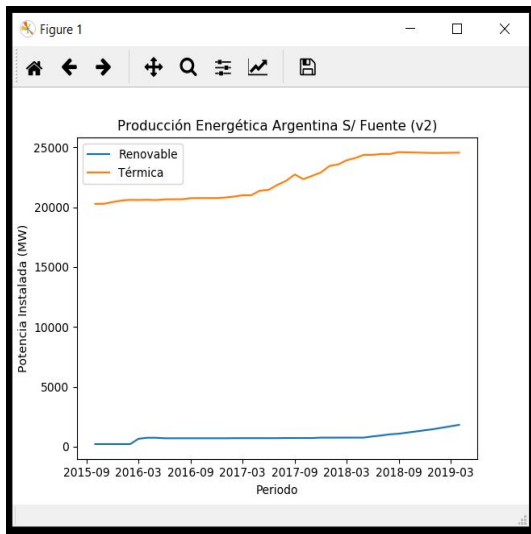
Graficar a la O-O: un gráfico

Para graficar al estilo OO:

```
1 >>> import matplotlib.pyplot as plt
2 >>> # creo el figura y axes
3 >>> fig, ax = plt.subplots() # crear objetos
4 >>> # defino series
5 >>> ax.plot(x1, y1, label="Renovable") # Serie 1
6 >>> ax.plot(x2, y2, label='Termica') # Serie 2
7 >>> # modifico labels
8 >>> ax.set_xlabel('Periodo')
9 >>> ax.set_ylabel('Potencia Instalada (MW)')
10 >>> ax.set_title("Produccion Energetica Argentina S/ Fuente
    (v2)")
11 >>> # agrego leyenda
12 >>> ax.legend()
13 >>> fig.show()
```

Nota: luego de correr `fig.show()` se abre una ventana con el gráfico. Es necesario cerrarla para seguir trabajando sobre la consola.

Graficar a la O-O: un gráfico (resultado)



Graficar a la pyplot: múltiples gráficos

Para armar una figura con dos (o más) gráficos:

```
1 >>> import matplotlib.pyplot as plt
2 >>> # ejemplo 2 ax en un fig
3 >>> plt.figure(figsize=(9, 5))
4 >>> # defino primer grafico
5 >>> plt.subplot(121)
6 >>> plt.plot(x1, y1)
7 >>> plt.title('Fuente Renovable')
8 >>> plt.xticks(rotation=45) # valores eje x en diagonal
9 >>> # defino segundo grafico
10 >>> plt.subplot(122)
11 >>> plt.plot(x2, y2)
12 >>> plt.title('Fuente Hidraulica')
13 >>> plt.xticks(rotation=45) # valores eje x en diagonal
14 >>> # definto titulo general
15 >>> plt.suptitle("Ejemplo dos gráficos en una figura")
16 >>> plt.show()
```

Graficar a la pyplot: dos gráficos (resultados)

